Resumen: Lenguaje Imperativo Simple (LIS)

1. Lenguaje Imperativo Simple (LIS)

1.1. Gramática abstracta

$$
\begin{array}{lcl}
\langle comm \rangle & ::= & \textbf{skip} \\
 & & \langle var \rangle := \langle intexp \rangle \\
 & & \langle comm \rangle \ ; \ \langle comm \rangle \\
 & & \textbf{if} \ \langle boolexp \rangle \ \textbf{then} \ \langle comm \rangle \ \textbf{else} \ \langle comm \rangle \\
 & & \textbf{newvar} \ \langle var \rangle := \langle intexp \rangle \ \textbf{in} \ \langle comm \rangle \\
 & & \textbf{while} \ \langle boolexp \rangle \ \textbf{do} \ \langle comm \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle intexp \rangle & ::= & \langle natconst \rangle \\
 & & \langle var \rangle \\
 & & \langle intexp \rangle + \langle intexp \rangle \\
 & & \langle intexp \rangle * \langle intexp \rangle \\
 & & \langle intexp \rangle - \langle intexp \rangle \\
 & & \langle intexp \rangle \ / \ \langle intexp \rangle \\
 & & \langle intexp \rangle \ \% \ \langle intexp \rangle \\
 & & - \langle intexp \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle boolexp \rangle & ::= & \langle boolconst \rangle \\
 & & \langle intexp \rangle = \langle intexp \rangle \\
 & & \langle intexp \rangle < \langle intexp \rangle \\
 & & \langle intexp \rangle \leq \langle intexp \rangle \\
 & & \langle intexp \rangle > \langle intexp \rangle \\
 & & \langle intexp \rangle \geq \langle intexp \rangle \\
 & & \neg \langle boolexp \rangle \\
 & & \langle boolexp \rangle \vee \langle boolexp \rangle \\
 & & \langle boolexp \rangle \wedge \langle boolexp \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle natconst \rangle & ::= & 0 \mid 1 \mid 2 \mid .... \\
\langle boolconst \rangle & ::= & \textbf{true} \mid \textbf{false}
\end{array}
$$

1.2. Estados

$$\Sigma = \mathrm{Var} \to \mathbb{Z}$$

Un estado es una función que asigna a cada variable un entero.

2. Funciones semánticas

$$
\begin{aligned}
[\![\cdot]\!]_{intexp} &: \langle intexp \rangle \to (\Sigma \to \mathbb{Z}) \\
[\![\cdot]\!]_{boolexp} &: \langle boolexp \rangle \to (\Sigma \to \{V, F\}) \\
[\![\cdot]\!]_{comm} &: \langle comm \rangle \to (\Sigma \to \Sigma)
\end{aligned}
$$

Todos los comandos terminan en su ejecución y su resultado.[es] un nuevo estado

3. Problema de la iteración

3.1. No terminación

Se introduce:

$$\Sigma_\perp = \Sigma \cup \{\perp\}$$

3.2. Nuevo tipo semántico

$$[\![\cdot]\!] : \langle comm \rangle \to (\Sigma \to \Sigma_\perp)$$

4. Extensión estricta

Dada una $f \in P \to D$, donde P es un predominio y D un dominio, se denota por $f_{\perp\perp}$ la única extensión estricta de f a $P_\perp$. Así, $f_{\perp\perp} \in P_\perp \to D$ está definida por:

$$f_{\perp\perp}\, x = \begin{cases} \perp & \text{si } x = \perp \\ f\, x & \text{si no} \end{cases}$$

5. Resumen de llaves semanticas

Expresiones Enteras

$[\![\cdot]\!] \in \langle intexp \rangle \to \Sigma \to \mathbb{Z}$

$$\begin{aligned} [\![0]\!]\sigma &= 0 \\ [\![-e]\!]\sigma &= -[\![e]\!]\sigma \\ [\![e_0 + e_1]\!]\sigma &= [\![e_0]\!]\sigma + [\![e_1]\!]\sigma \quad \text{(similarmente para } -, *, \dots) \\ [\![v]\!]\sigma &= \sigma\, v \end{aligned}$$

Expresiones Booleanas

$[\![\cdot]\!] \in \langle boolexp \rangle \to \Sigma \to \mathbb{B}$

$$\begin{aligned} [\![\text{true}]\!]\sigma &= V \\ [\![\text{false}]\!]\sigma &= F \\ [\![e_0 = e_1]\!]\sigma &= [\![e_0]\!]\sigma = [\![e_1]\!]\sigma \quad \text{(similarmente para } <, >, \dots) \\ [\![\neg b]\!]\sigma &= \neg[\![b]\!]\sigma \\ [\![b_0 \wedge b_1]\!]\sigma &= [\![b_0]\!]\sigma \wedge [\![b_1]\!]\sigma \quad \text{(similarmente para } \vee, \dots) \end{aligned}$$

Comandos

$[\![\cdot]\!] \in \langle comm \rangle \to \Sigma \to \Sigma_\perp$

$$
\begin{aligned}
[\![\text{skip}]\!]\sigma &= \sigma \\
[\![v := e]\!]\sigma &= [\sigma | v : [\![e]\!]\sigma] \\
[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!]\sigma &= \begin{cases} [\![c_0]\!]\sigma & \text{si } [\![b]\!]\sigma \\ [\![c_1]\!]\sigma & \text{si no} \end{cases} \\
[\![c_0; c_1]\!]\sigma &= [\![c_1]\!]_\perp([\![c_0]\!]\sigma) \\
[\![\text{newvar } v := e \text{ in } c]\!]\sigma &= (\lambda\sigma' \in \Sigma.[\sigma' | v : \sigma\, v])_\perp([\![c]\!][\sigma | v : [\![e]\!]\sigma]) \\
[\![\text{while } b \text{ do } c]\!] &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \to \Sigma_\perp}
\end{aligned}
$$

donde:

$$
\begin{aligned}
f_{\perp\perp} x &= \begin{cases} \perp & \text{si } x = \perp \\ f\, x & \text{si no} \end{cases} \\
F\, w\, \sigma &= \begin{cases} w_{\perp\perp}([\![c]\!]\sigma) & \text{si } [\![b]\!]\sigma \\ \sigma & \text{si no} \end{cases}
\end{aligned}
$$

6. Propiedades

6.1. Variables libres

$$
\begin{aligned}
FV(\texttt{skip}) &= \emptyset \\
FV(v := e) &= \{v\} \cup FV(e) \\
FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\
FV(\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \\
FV(\texttt{while } b \texttt{ do } c) &= FV(b) \cup FV(c) \\
FV(\texttt{newvar } v := e \texttt{ in } c) &= FV(e) \cup (FV(c) - \{v\})
\end{aligned}
$$

6.2. Variables asignables

$$
\begin{aligned}
FA(\texttt{skip}) &= \emptyset \\
FA(v := e) &= \{v\} \\
FA(c_0; c_1) &= FA(c_0) \cup FA(c_1) \\
FA(\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1) &= FA(c_0) \cup FA(c_1) \\
FA(\texttt{while } b \texttt{ do } c) &= FA(c) \\
FA(\texttt{newvar } v := e \texttt{ in } c) &= FV(c) - \{v\}
\end{aligned}
$$

6.3. Teorema de Coincidencia

1. $(\forall w \in FV(c).\sigma w = \sigma' w)$ implica,
   - o bien $[\![c]\!]\sigma = \bot = [\![c]\!]\sigma'$,
   - o bien $[\![c]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in FV(c).[\![c]\!]\sigma w = [\![c]\!]\sigma' w$
2. Si $[\![c]\!]\sigma \neq \bot$, entonces $\forall w \notin FA(c).[\![c]\!]\sigma w = \sigma w$. (esto es intangibilidad)

6.4. Teorema de Renombre

Otra propiedad que cobra importancia en el lenguaje imperativo es la que establece que no importa el nombre de las variables utilizadas en las declaraciones de variables locales (o sea las ligadas):

$$u \notin FV(c) - \{v\} \Rightarrow [\![\textbf{newvar } u := e \textbf{ in } c[v \to u]]\!] = [\![\textbf{newvar } v := e \textbf{ in } c]\!]$$

7. Sustituciones

Conjunto de sustituciones

$$\Delta = \mathrm{Var} \to \mathrm{Var}$$

Las ecuaciones que la definían para la lógica de predicados siguen valiendo (salvo el caso de los cuantificadores, que ahora no tenemos). Ahora se agregan ecuaciones para los comandos.

$$_/_ \in \langle comm \rangle \times \Delta \to \langle comm \rangle$$

$$\begin{aligned}
\mathbf{skip}/\delta &= \mathbf{skip} \\
(v := e)/\delta &= (\delta v) := (e/\delta) \\
(c_0; c_1)/\delta &= (c_0/\delta); (c_1/\delta) \\
(\mathbf{if}\ b\ \mathbf{then}\ c_0\ \mathbf{else}\ c_1)/\delta &= \mathbf{if}\ b/\delta\ \mathbf{then}\ c_0/\delta\ \mathbf{else}\ c_1/\delta \\
(\mathbf{while}\ b\ \mathbf{do}\ c)/\delta &= \mathbf{while}\ b/\delta\ \mathbf{do}\ c/\delta
\end{aligned}$$

El caso del **newvar** tiene los mismos inconvenientes que los cuantificadores, ahora es un poco más simple ya que δw son siempre variables.

$$(\mathbf{newvar}\ v := e\ \mathbf{in}\ c)/\delta = \mathbf{newvar}\ v_{new} := e/\delta\ \mathbf{in}\ (c/[\delta|v : v_{new}])$$

donde $v_{new} \notin \{\delta w | w \in FV(c) - \{v\}\}$

7.1. Teorema de Sustitución

Si δ es inyectiva sobre $FV(c)$ y $\forall w \in FV(c).\ \sigma(\delta w) = \sigma' w$, entonces

- o bien $[\![c/\delta]\!]\sigma = \bot = [\![c]\!]\sigma'$,
- o bien $[\![c/\delta]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in FV(c).\ [\![c/\delta]\!]\sigma(\delta w) = [\![c]\!]\sigma' w$.

7.2. Lema de Sustitución

Sea $FV(c) \subseteq V \subseteq \langle var \rangle$ tal que δ es inyectiva sobre V y $\forall w \in V.\ \sigma(\delta w) = \sigma' w$. Entonces,

- o bien $[\![c/\delta]\!]\sigma = \bot = [\![c]\!]\sigma'$,
- o bien $[\![c/\delta]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in V.\ [\![c/\delta]\!]\sigma(\delta w) = [\![c]\!]\sigma' w$.

8. Fallas en el Lenguaje Imperativo Simple

Para modelar la transferencia de control por fallas, se incorporan excepciones al lenguaje imperativo mediante nuevas construcciones:

$$\langle comm \rangle ::= fail \mid catchin\ \langle comm \rangle\ with\ \langle comm \rangle$$

8.1. Dominio semántico

Se extiende el conjunto de estados:

$$\Sigma' = \Sigma \cup (\{abort\} \times \Sigma)$$

La función semántica queda definida como:

$$[[_]] : \langle comm \rangle \to (\Sigma \to \Sigma'_\bot)$$

donde $\Sigma'_\bot$ es un dominio llano con elemento indefinido $\bot$.

8.2. Semántica de los comandos básicos

$$
\begin{aligned}
[[skip]]\sigma &= \sigma \\
[[fail]]\sigma &= \langle abort, \sigma \rangle \\
[[v := e]]\sigma &= \sigma[v \mapsto [[e]]\sigma]
\end{aligned}
$$

8.3. Operadores de transferencia de control

Para definir la semántica de composición y manejo de excepciones, se introducen tres operadores:

- **Extensión estricta (f_*):** Propaga fallas (no continúa la ejecución).

$$
f_*(x) = \begin{cases} f(\sigma) & \text{si } x = \sigma \in \Sigma \\ x & \text{si } x \text{ es abort o } \bot \end{cases}
$$

- **Extensión de manejo (f_+):** Se ejecuta solo si hubo falla.

$$
f_+(x) = \begin{cases} f(\sigma) & \text{si } x = \langle abort, \sigma \rangle \\ x & \text{en otro caso} \end{cases}
$$

- **Extensión total ($f_\dagger$):** Se ejecuta siempre, preservando el estado de falla (en este caso se hace asi ya que queremos que en el newvar se ejecute la función para restaurar el estado original).

$$
f_\dagger(x) = \begin{cases} \langle abort, f(\sigma) \rangle & \text{si } x = \langle abort, \sigma \rangle \\ f(x) & \text{si } x \in \Sigma \\ \bot & \text{si } x = \bot \end{cases}
$$

8.4. Semántica de las construcciones extendidas

$$
[[c_0; c_1]]\sigma = [[c_1]]_*([[c_0]]\sigma)
$$

$$
[[catchin\ c_0\ with\ c_1]]\sigma = [[c_1]]_+([[c_0]]\sigma)
$$

$$
[[newvar\ v := e\ in\ c]]\sigma = (\lambda\sigma'.\sigma'[v \mapsto \sigma(v)])_\dagger([[c]](\sigma[v \mapsto [[e]]\sigma]))
$$

$$
[[while\ b\ do\ c]] = \bigsqcup_{i=0}^{\infty} F^i(\bot)
$$

donde

$$
F(w)(\sigma) = \begin{cases} w_*([[c]]\sigma) & \text{si } [[b]]\sigma \\ \sigma & \text{en otro caso} \end{cases}
$$

9. Entrada/Salida y Manejo de Errores

9.1. Sintaxis de Comandos

$$
\begin{array}{rcl}
\langle comm \rangle & ::= & \text{skip} \mid \langle var \rangle := \langle intexp \rangle \mid \langle comm \rangle ; \langle comm \rangle \\
& \mid & \text{if } \langle boolexp \rangle \text{ then } \langle comm \rangle \text{ else } \langle comm \rangle \\
& \mid & \text{newvar } \langle var \rangle := \langle intexp \rangle \text{ in } \langle comm \rangle \\
& \mid & \text{while } \langle boolexp \rangle \text{ do } \langle comm \rangle \\
& \mid & \text{fail} \mid \text{catchin } \langle comm \rangle \text{ with } \langle comm \rangle \\
& \mid & !\langle intexp \rangle \mid ?\langle var \rangle
\end{array}
$$

9.2. Dominio de Salidas (Ω)

El dominio de salidas Ω se define como el conjunto de posibles resultados de la ejecución de un programa, incluyendo terminación exitosa, error o divergencia (ejecución infinita).

$$
\begin{aligned}
\Omega = \quad & \{\langle n_0, \ldots, n_k, \sigma' \rangle \mid n_i \in \mathbb{Z}, \sigma' \in \hat{\Sigma}\} \\
& \cup \{\langle n_0, \ldots, n_k \rangle \mid n_i \in \mathbb{Z}\} \\
& \cup \{\langle n_0, n_1, n_2, n_3, \ldots, n_k, \ldots \rangle \mid n_i \in \mathbb{Z}\}
\end{aligned}
$$

Donde el conjunto de estados extendido $\hat{\Sigma}$ se define como:

$$\hat{\Sigma} = \Sigma \cup (\{\mathbf{abort}\} \times \Sigma)$$

9.3. Concatenación de tuplas

Podemos "concatenar" ciertos elementos de Ω para obtener nuevos elementos del mismo dominio:

$$\langle n_0, \ldots, n_k \rangle +\!\!+ \langle m_0, \ldots, s \rangle = \langle n_0, \ldots, n_k, m_0, \ldots, s \rangle$$

Aquí, s puede representar tanto un número entero como un estado (finalizado o fallido).

9.4. Estructura del Dominio

Para tratar la semántica de forma denotacional, definimos una estructura de orden sobre Ω:

1. **Orden de prefijo:** El orden está dado por la relación de prefijo. Por ejemplo:
$$\langle 4, 3, 2, 1 \rangle \sqsubseteq \langle 4, 3, 2, 1, \sigma \rangle$$

2. **Supremo de cadenas no interesantes:** Es simplemente el último elemento de la cadena.

3. **Supremo de cadenas interesantes:** Es la tupla (posiblemente infinita) que coincide en todas las posiciones de cada tupla de la cadena.

Observaciones:

- El elemento mínimo está dado por la tupla vacía: $\langle\rangle$.
- En una cadena "interesante" de tuplas, existen tuplas de cada longitud, por lo tanto, el supremo es efectivamente único.

9.5. Semántica de Programas

La nueva semántica de los comandos se define como una función que toma un estado inicial y devuelve un elemento del dominio de salidas:

$$[\![_]\!] : \langle comm \rangle \to (\Sigma \to \Omega)$$

9.6. Reglas semánticas

Para el comando de salida de datos ($!e$):

$$[\![!e]\!]\sigma = \langle [\![e]\!]\sigma, \sigma \rangle$$

$$[\![\mathbf{skip}]\!]\sigma = \langle \sigma \rangle$$

$$[\![v := e]\!]\sigma = \langle \sigma[v : [\![e]\!]\sigma] \rangle$$

$$[\![\mathbf{if}\ b\ \mathbf{then}\ c\ \mathbf{else}\ c']\!]\sigma = \begin{cases} [\![c]\!]\sigma & \text{si } [\![b]\!]\sigma \\ [\![c']\!]\sigma & \text{si } \neg[\![b]\!]\sigma \end{cases}$$

$$[\![c_0; c_1]\!]\sigma = [\![c_1]\!]_*([\![c_0]\!]\sigma)$$

$$[\![\mathbf{newvar}\ v := e\ \mathbf{in}\ c]\!]\sigma = (rest_{v,\sigma})_\dagger\,([\![c]\!][\sigma \mid v : [\![e]\!]\sigma])$$

9.7. Semántica del Bucle (While)

$$F_{b,c} : (\Sigma \to \Sigma') \to (\Sigma \to \Sigma')$$

$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg[\![b]\!]\sigma \\ f_*([\![c]\!]\sigma) & \text{si } [\![b]\!]\sigma \end{cases}$$

$$[\![\mathbf{while}\ b\ \mathbf{do}\ c]\!]\sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i\ \bot \right) \sigma$$

Nota: No es necesario cambiar la estructura de la ecuación, solo redefinir adecuadamente los operadores ($*$, $\dagger$, $\sqcup$) sobre el nuevo dominio Ω.

10. Funciones de Propagación de Errores

Estas funciones definen cómo se comportan las transformaciones de estado cuando encuentran estados de error o terminación.

10.1. Propagación estándar $(_)_*$

La función $(_)_*$ extiende una función de estado a una que maneja errores y abortos:

$$(_)_* : (\Sigma \to \Sigma') \to (\Sigma' \to \Sigma')$$

$$\begin{aligned} f_*(\bot) &= \bot \\ f_*(\langle \mathbf{abort}, \sigma \rangle) &= \langle \mathbf{abort}, \sigma \rangle \\ f_*(\sigma) &= f(\sigma) \end{aligned}$$

10.2. Propagación tipo Map $(_)_\dagger$

Similar a la función `map` en listas, aplica una transformación incluso en estados de aborto:

$$(_)_\dagger : (\Sigma \to \Sigma) \to (\Sigma' \to \Sigma')$$

$$\begin{aligned} f_\dagger(\bot) &= \bot \\ f_\dagger(\langle \mathbf{abort}, \sigma \rangle) &= \langle \mathbf{abort}, f(\sigma) \rangle \\ f_\dagger(\sigma) &= f(\sigma) \end{aligned}$$

11. Revisión de Funciones sobre Ω

Al trabajar con el dominio de secuencias Ω, las funciones de propagación se redefinen para operar sobre tuplas de enteros y estados.

11.1. Propagación en Ω

$$(_)_* : (\Sigma \to \Omega) \to (\Omega \to \Omega)$$

$$\begin{aligned} f_*(\langle n_0, \ldots, n_{k-1} \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \\ f_*(\langle n_0, \ldots, n_{k-1}, \sigma \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \mathbin{+\!\!+} f(\sigma) \\ f_*(\langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle) &= \langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle \\ f_*(\langle n_0, \ldots, n_{k-1}, \ldots \rangle) &= \langle n_0, \ldots, n_{k-1}, \ldots \rangle \end{aligned}$$

11.2. Función Map en Ω

$$(_)_\dagger : (\Sigma \to \Sigma) \to (\Omega \to \Omega)$$

$$\begin{aligned}
f_\dagger(\langle n_0, \ldots, n_{k-1} \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \\
f_\dagger(\langle n_0, \ldots, n_{k-1}, \sigma \rangle) &= \langle n_0, \ldots, n_{k-1}, f(\sigma) \rangle \\
f_\dagger(\langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle) &= \langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, f(\sigma) \rangle \rangle \\
f_\dagger(\langle n_0, \ldots, n_{k-1}, \ldots \rangle) &= \langle n_0, \ldots, n_{k-1}, \ldots \rangle
\end{aligned}$$

12. Funciones Inyectoras y Constructores

Para facilitar la definición recursiva de la semántica, consideramos las siguientes funciones auxiliares:

$$\begin{aligned}
\iota_\perp &: \{\langle\rangle\} \to \Omega & \iota_{term} &: \Sigma \to \Omega \\
\iota_\perp(\langle\rangle) &= \langle\rangle & \iota_{term}(\sigma) &= \langle \sigma \rangle \\
\iota_{abort} &: \Sigma \to \Omega & \iota_{out} &: \mathbb{Z} \times \Omega \to \Omega \\
\iota_{abort}(\sigma) &= \langle \sigma \rangle & \iota_{out}(\langle n, \omega \rangle) &= \langle n \rangle \mathbin{+\!\!+} \omega
\end{aligned}$$

12.1. Propagación de errores (Definición Alternativa)

Utilizando los constructores anteriores, la propagación f_* se puede ver como:

$$\begin{aligned}
f_*(\perp) &= \perp \\
f_*(\iota_{term}(\sigma)) &= f(\sigma) \\
f_*(\iota_{abort}(\sigma)) &= \iota_{abort}(\sigma) \\
f_*(\iota_{out}(\langle n, \omega \rangle)) &= \iota_{out}(\langle n, f_*(\omega) \rangle)
\end{aligned}$$

Observación: Para asegurar que el mínimo punto fijo exista y sea único, es fundamental que la función f_* sea **continua**.